

PROGRAMMER EN C/C++ SOUS GNU/LINUX

COMPILATION : RAPPEL

prog.c (fichier source) $\xrightarrow{\text{compilation}} \xrightarrow{\text{Édition des liens}}$ prog.o (fichier objet) $\xrightarrow{\text{Édition des liens}}$ prog (exécutable)

- ✓ La **compilation** en fichiers objets laisse l'identification de certains symboles à plus tard. Avant de pouvoir exécuter ces fichiers objets, il faut résoudre les symboles et les lier à une bibliothèque. Le lien peut être :
 - 👉 statique : le fichier objet et la bibliothèque sont liés dans le même fichier exécutable.
 - 👉 dynamique : le fichier objet est lié avec la bibliothèque, mais pas dans le même fichier exécutable ; les liens sont établis lors du lancement de l'exécutable.
- ✓ L'**édition des liens** est un processus qui permet de créer des fichiers exécutables ou des bibliothèques dynamiques ou statiques, à partir de fichiers objets.
- ✓ La phase d'**édition des liens** consiste en la construction d'une image mémoire contenant l'ensemble des parties de code compilées séparément (modules, sous-programmes ou bibliothèques de sous-programmes).

COMPILER AVEC GCC

- ✓ Les compilateurs disponibles sur les systèmes Linux font tous partie de la GNU Compiler Collection, plus communément appelée GCC (<http://gcc.gnu.org/>).
 - ☞ un fichier source C++ (*reciprocal.cpp*) et ;
 - ☞ un fichier source C (*main.c*)
 - ☞ Un fichier d'entête (*reciprocal.hpp*)
- ✓ GCC inclut également des compilateurs C, C++, Java, Objective-C, Fortran etc.
- ✓ Supposons que vous ayez un programme *reciprocal* avec :
 - ☞ un fichier source C++ (*reciprocal.cpp*) et ;
 - ☞ un fichier source C (*main.c*)
 - ☞ Un fichier d'entête (*reciprocal.hpp*)
- ✓ Ce programme calcule l'inverse d'un entier.

EXEMPLE

Fichier source C main.c

```
#include <stdio.h>
#include <stdlib.h>
#include "reciprocal.hpp"
int main (int argc, char **argv) {
    int i;
    i = atoi (argv[1]);
    printf ("L'inverse de %d est %g\n", i, reciprocal (i));
    return 0;
}
```

Fichier source C++ reciprocal.cpp

```
#include <cassert>
#include "reciprocal.hpp"
double reciprocal (int i) {
    assert (i != 0); // i doit être différent de zéro
    return 1.0/i;
}
```

Fichier d'entête reciprocal.hpp

```
#ifndef __cplusplus
extern "C" {
#endif
extern double reciprocal (int i);
#ifdef __cplusplus
}
#endif
```

COMPILER UN FICHIER SOURCE ISOLÉ

- ✓ L'option `-c` indique à gcc ou `g++` de ne compiler le fichier que sous forme d'un fichier objet; sinon, gcc ou `g++` tenterait de lier le programme afin de produire un exécutable.

```
% gcc -c main.c    → main.o  
% g++ -c reciprocal.cpp
```

- ✓ Maintenant que vous avez compilé `main.c` et `reciprocal.cpp`, vous devez les lier.

```
% g++ -o reciprocal main.o reciprocal.o
```

- ✓ Vous pouvez maintenant lancer `reciprocal` comme ceci:

```
% ./reciprocal 7
```

- ✓ L'option `-I` est utilisée pour indiquer à GCC où rechercher les fichiers d'entête.

```
% g++ -c -I ../include reciprocal.cpp  
% g++ -c -D NDEBUG reciprocal.cpp
```

- ✓ Désactivez la vérification en définissant la macro `NDEBUG`.

LIER LES FICHIERS OBJET

- ✓ L'option `-O` optimise le code afin qu'il s'exécute aussi rapidement que possible

```
% g++ -c -O2 reciprocal.cpp
```

- ✓ Ajout d'une bibliothèque (`-l`)

```
% g++ -o reciprocal main.o reciprocal.o -lpam
```

→ inclure *libpam.a* lors de l'édition de liens

- ✓ Si vous voulez que l'éditeur de liens recherche en plus dans d'autres répertoires, vous devez utiliser l'option `-L`,

```
% g++ -o reciprocal main.o reciprocal.o -  
L/usr/local/lib/pam -lpam
```

→ indiquer à l'éditeur de liens de rechercher les bibliothèques dans le répertoire `/usr/local/lib/pam`

```
% gcc -o app app.o -L. -ltest
```

→ indiquer à l'éditeur de liens de rechercher la bibliothèque `test` dans le répertoire courant

AUTOMATISER LE PROCESSUS AVEC GNU MAKE

- ✓ Objectifs :
 - ☞ Compiler de gros programmes dont les sources sont réparties en plusieurs fichiers.
 - ☞ Déterminer quelles sont les parties qui doivent être recompilées lorsque des modifications ont été effectuées et appeler les commandes nécessaires à leur recompilation.
 - ☞ La commande **make** interprète pour cela un fichier, qu'elle recherche dans le répertoire courant, appelé **Makefile** et qui décrit les relations entre fichiers sources et contient les commandes nécessaires à la compilation.
- ✓ Les makefiles sont, en grande partie, constitués d'une suite de règles. Une règle (rule en anglais) est constituée de :
 - ☞ cibles (targets en anglais);
 - ☞ prérequis ou dépendances (prerequisites ou dependencies en anglais);
 - ☞ commandes (commands en anglais).
- ✓ Une cible est généralement le nom de l'un des fichiers (ou d'un des ensembles de fichiers) que l'on souhaite construire avec le makefile.
- ✓ Les prérequis désignent toujours les fichiers dont dépendent la cible ou les cibles. Plus précisément, un fichier Prq donné doit être le prérequis d'une cible Targ donné, si on considère qu'il faut remettre à jour Targ à chaque fois que le fichier Prq est modifié.
- ✓ Enfin les commandes sont une liste de commandes qu'il faut exécuter pour reconstruire la cible.

AUTOMATISER LE PROCESSUS AVEC GNU MAKE

- ✓ Exemple précédant : fichier Makefile

```
reciprocal: main.o reciprocal.o
    g++ $(CFLAGS) -o reciprocal main.o reciprocal.o
main.o: main.c reciprocal.hpp
    gcc $(CFLAGS) -c main.c
reciprocal.o: reciprocal.cpp reciprocal.hpp
    g++ $(CFLAGS) -c reciprocal.cpp
clean:
    rm -f *.o reciprocal
```

☞ % make

- ✓ Recompiler avec les optimisations activées, vous procéderiez de la façon suivante:

☞ % make clean

☞ rm -f *.o reciprocal

☞ % make CFLAGS=-O2

☞ gcc -O2 -c main.c

☞ g++ -O2 -c reciprocal.cpp

☞ g++ -O2 -o reciprocal main.o reciprocal.o

☞ Notez que le drapeau -O2 a été inséré à la place de \$(CFLAGS) dans les règles.

LE CONFIGURATEUR MAKE

- ✓ Lit une spécification de dépendances ou mode d'emploi de fabrication, et l'interprète pour créer une nouvelle version complète.
- ✓ Il vérifie les dates de dernière modification, pour réaliser le minimum de compilation.
- ✓ **make** examine si les dépendances de l'entrée sont à jour; sinon, fait exécuter par des processus UNIX les commandes de fabrication en les soumettant une par une à l'interpréteur shell.

MODE DE FONCTIONNEMENT: MAKEFILE

- ✓ Syntaxe de Makefile :
- ✓ Lignes de commentaires débutent par un #
- ✓ Définitions d'entrée nomment les entrées et définissent les dépendances:
 - ☞ Cible : liste de dépendances
 - ☞ Exemple:
 - ☞ **prog: def.c sp.o prog.c**
- ✓ Règles de fabrication forment la recette; suivent une définition d'entrée:
 - ☞ <tab> <commande>
 - ☞ Exemple: **cc -o prog prog.o def.o sp.o**
- ✓ Les entrées génériques
 - ☞ .s [.t]+ :
 - ☞ .s.t. mode d'emploi général à appliquer pour fabriquer un fichier de suffixe .t quand on a un fichier de suffixe .s
- ✓ Les lignes d'inclusion include <fichier>
- ✓ Définitions de macros une macro est une variable de make
 - ☞ <nom macro> = <texte définissant la macro>

L'APPEL DE MAKE

- ✓ **make** [-f makefile] [options] [macro=valeur] [cible]
- ☞ -i Ignorer les erreurs retournées par les commandes exécutées;
 - ☞ -s Ne pas imprimer les commandes avant de les exécuter;
 - ☞ -r Ne pas utiliser les règles implicites;
 - ☞ -t Maj des dates des fichiers cibles sans exécuter les commandes;
 - ☞ -o Retourner un code précisant l'état de la cible
 - ☞ -f fic fic est le nom du fichier description à utiliser (makefile).

UTILISATION DES MACROS

- ✓ La substitution de macro précédant son nom entre () par \$.

```
MISEAUPPOINT = # rien: pas de mise au point
```

```
prog:    def.p sp.o  prog.o
```

```
    pc -o prog prog.o def.o sp.o
```

```
def.o:   def.p
```

```
    pc -c $(MISEAUPPOINT) def.p
```

```
sp.o:    sp.p  def.o
```

```
    pc -c $(MISEAUPPOINT) sp.p
```

```
prog.o:  prog.p      def.o
```

```
    pc -c $(MISEAUPPOINT) prog.p
```

- ✓ On obtient un module prog pour la mise au point de 2 façons:

☞ MISEAUPPOINT = -g # dans le makefile

☞ \$ make MISEAUPPOINT = -g # dans l'appel de make

EXERCICE 1

- ✓ Ecrire un programme `main.c` qui fait appel à diverses fonctions déclarées dans un fichier `liste.c` (et `liste.h` par conséquent) et permettant de réaliser des fonctions arithmétiques usuelles. Selon les paramètres entrés par l'utilisateur (`nb1`, `nb2`, opérateur), une des fonctions sera exécutées et son résultat affiché.
- ✓ Ecrire un fichier `Makefile` permettant d'automatiser la compilation de ce programme.

SOLUTION

Liste.c :

```
#include <stdio.h>
```

```
int somme(int nb1, int nb2)
```

```
{
```

```
int som;
```

```
som=nb1+nb2;
```

```
return som;
```

```
}
```

```
int produit(int nb1, int nb2)
```

```
{
```

```
int prod;
```

```
prod=nb1*nb2;
```

```
return prod;
```

```
}
```

Liste.h

```
#ifndef _LISTE_H
```

```
#define _LISTE_H
```

```
void affiche(void);
```

```
int somme(int nb1, int nb2);
```

```
int produit(int nb1, int nb2);
```

```
#endif
```

main.c

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include "liste.h"
```

```
void main(int argc, char* argv[])
```

```
{
```

```
if (strcmp(argv[3],"+")==0)
```

```
printf("%d\n",somme(atoi(argv[1]),atoi(argv[2])));
```

```
if (strcmp(argv[3],"*")==0)
```

```
printf("%d\n",produit(atoi(argv[1]),atoi(argv[2])));
```

```
}
```

SOLUTION

✓ Makefile :

```
main : main.o liste.o
    gcc -o main main.o liste.o
liste.o : liste.c liste.h
    gcc -c  liste.c
main.o   : main.c liste.h
    gcc -c  main.c liste.h

clean :
    rm main main.o liste.o
```

GNU DEBUGGER : INTRODUCTION

- ✓ Le débogueur est utilisé pour trouver pourquoi un programme ne se comporte pas comme prévu.
- ✓ Compiler avec les Informations de Débogage
- ✓ Pour compiler en activant les informations de débogage, utiliser l'option -g
 - 👉 **% make CFLAGS=-g**
 - 👉 gcc -g -c main.c
 - 👉 g++ -g -c reciprocal.cpp
 - 👉 g++ -g -o reciprocal main.o reciprocal.o

LANCER GDB

```
% gdb reciprocal
```

```
(gdb)
```

```
lancer votre programme au sein du débogueur.
```

```
(gdb) run
```

```
Starting program: reciprocal
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
%%__%strtol_internal (nptr=0x0, endptr=0x0, base=10, group=0)
```

```
at strtol.c:287
```

```
287      strtol.c: No such file or directory.
```

```
(gdb)
```

```
Vous pouvez observer la pile en utilisant la commande where:
```

```
(gdb) where
```

```
#0 %%__%strtol_internal (nptr=0x0, endptr=0x0, base=10, group=0)
```

```
at strtol.c:287
```

```
#1 0x40096fb6 in atoi (nptr=0x0) at ../stdlib/stdlib.h:251
```

```
#2 0x804863e in main (argc=1, argv=0xbffff5e4) at main.c:8
```

```
main a appelé la fonction atoi avec un pointeur NULL ce qui est la  
source de l'erreur.
```

LES COMMANDES DE GDB

remonter de deux niveaux dans la pile jusqu'à atteindre main

```
(gdb) up 2
```

```
#2 0x804863e in main (argc=1, argv=0xbffff5e4) at main.c:8
```

```
8          i = atoi (argv[1]);
```

GDB retrouver le fichier source main.c et il affiche la ligne contenant l'appel

Vous pouvez inspecter la valeurs des variables en utilisant la commande print:

```
(gdb) print argv[1]
```

```
$2 = 0x0
```

Cela confirme que le problème vient d'un pointeur NULL passé à atoi.

Vous pouvez placer un point d'arrêt en utilisant la commande break:

```
(gdb) break main
```

```
Breakpoint 1 at 0x804862e: file main.c, line 8.
```

relancer le programme avec un argument, comme ceci:

```
(gdb) run 7
```

```
Starting program: reciprocal 7
```

```
Breakpoint 1, main (argc=2, argv=0xbffff5e4) at main.c:8
```

```
8          i = atoi (argv[1])
```

Vous remarquez que le débogueur s'est arrêté au niveau du point d'arrêt.

LES COMMANDES DE GDB

Vous pouvez passer à l'instruction se trouvant après l'appel à `atoi` en utilisant la commande `next`:

```
(gdb) next
```

```
9  printf ("L'inverse de %d est %g\n", i, reciprocal  
    (i));
```

Si vous voulez voir ce qui se passe à l'intérieur de la fonction `reciprocal`, utilisez la commande `step`,

```
(gdb) step
```

```
reciprocal (i=7) at reciprocal.cpp:6
```

```
6      assert (i != 0);
```

Vous êtes maintenant au sein de la fonction `reciprocal`.

LES COMMANDES DE GDB - RÉSUMÉ

Commande	Effet
run	lance le programme (s'arrête au prochain point d'arrêt)
continue	relance le programme (s'arrête au prochain point d'arrêt)
break yyy.c:xx	place un point d'arrêt à la ligne xx du fichier yyy.c (ou le nom d'une fonction)
info breakpoints	liste les points d'arrêts
delete	efface les points d'arrêts
next	exécute une instruction (ne rentre pas dans les fonctions)
step	exécute une instruction (rentre potentiellement dans les fonctions)
finish	exécute les instructions jusqu'à la sortie de la fonction
until xx	exécute les instructions jusqu'à la ligne xx

EXERCICE 2

- ✓ Recompiler l'exercice 1 en utilisant l'option `-g`
- ✓ Exécutez gdb, insérez un point d'arrêt (break) sur la fonction main puis exécutez le programme main étape par étape.
- ✓ Visualisez à chaque étape la valeur de la variable som ou prod.

LA LISTE D'ARGUMENTS

INTERACTION AVEC L'ENVIRONNEMENT D'EXÉCUTION

- ✓ En lançant un programme, vous pouvez passer plus d'informations en ajoutant un ou plusieurs mots après le nom du programme
- ✓ Ce sont des arguments de ligne de commande ou liste d'arguments du programme.
- ✓ Exemple :
- ✓ `% ls -s /`
 - ☞ le programme `ls` reçoit trois éléments :
 - le nom du programme lui-même, (`ls`)
 - Les second et troisième élément sont les deux arguments de ligne de commande, `-s` et `/`.
- ✓ La fonction `main` de votre programme peut accéder à la liste d'arguments via ses paramètres **`argc`** et **`argv`**
 - ☞ **`argc`**, indique le nombre d'éléments dans la liste.
 - ☞ **`argv`**, est un tableau de pointeurs sur des caractères (taille est `argc`,) qui pointent vers les éléments de la liste d'arguments, qui sont des chaînes terminées par zéro.

UTILISER ARGC ET ARGV

✓ arglist.c

```
#include <stdio.h>
int main (int argc, char* argv[])
{
    printf ("Le nom de ce programme est '%s'.\n", argv[0]);
    printf ("Ce programme a été invoqué avec %d arguments.\n",
argc - 1);
    /* A-t-on spécifié des arguments sur la ligne de commande ? */
    if (argc > 1) {
        /* Oui, les afficher. */
        int i;
        printf ("Les arguments sont :\n");
        for (i = 1; i < argc; ++i)
            printf (" %s\n", argv[i]);
    }
    return 0;
}
```

LA GESTION DES ERREURS : PERROR()

- ✓ Un appel système retourne une valeur différente de 0 si une erreur survient
 - ☞ “**errno**”, est une variable «système» qui contient le code numérique de l’erreur du dernier appel système, un appel système qui échoue modifie la valeur de errno
 - ☞ La liste globale d'erreurs **sys_errlist[]** indexée par errno peut être utilisée pour obtenir le message d'erreur (errno < sys_nerr).
 - ☞ “**perror(char *str)**” est une fonction standard C qui décrit les erreurs des appels systèmes (affiche str suivie par “:” et une description de la dernière erreur rencontrée, sinon “Error 0”)
 - ☞ **char *strerror (int errnum)** Obtenir le libellé d'un code d'erreur « errnum »
- ✓ “/usr/include/errno.h” contient une liste des codes d’erreurs prédéfinies
- ✓ Exemples:

```
#define EPERM 1 /* Not owner */  
#define ENOENT 2 /* No such file or directory */  
#define ESRCH 3 /* No such process */  
#define EINSR 4 /* Interrupted System call */  
#define EIO 5 /* I/O error */
```


LA GESTION DES ERREURS : PERROR()

```
#include <stdio.h>
#include <sys/file.h>
#include <errno.h>
main()
{
    int fd;
    /* Open a nonexistent file to cause an error */
    fd = open("nonexist.txt", O_RDONLY);
    if ( fd== -1 ) { /* fd == -1 , an error occurred */
        printf( "errno = %d \n", errno );
        perror("main");
    }
    fd=open( "/", O_WRONLY ); /* Force a different error */
    if ( fd== -1 ) {
        printf("errno=%d\n", errno);
        perror("main");
    } /* Execute a successful system call */
    fd = open("nonexist.txt", O_RDONLY | O_CREAT, 0644 );
    printf("errno=%d\n", errno); /* Display after successful call */
    perror("main");
    errno=0; /* Manually reset error variable */
    perror("main");
}
```

ASSERT

- ✓ Détecter des conditions inattendues

👉 `assert(condition)`

- ✓ Le programme s'arrête si l'expression est fausse, après avoir affiché un message d'erreur (nom du fichier, le numéro de ligne et le texte de l'expression)

```
for (i = 0; i < 100; ++i)
    assert (do_something () == 0);
```

Ou mieux,

```
for (i = 0; i < 100; ++i) {
    int status = do_something ();
    assert (status == 0);
}
```

LES BIBLIOTHÈQUES STATIQUES

- ✓ Une archive (ou bibliothèque statique) est une collection de fichiers objets stockée dans un seul fichier objet.
- ✓ Lorsque vous fournissez une archive à l'éditeur de liens, il recherche au sein de cette archive les fichiers dont il a besoin, les extrait et les lie avec votre programme comme si vous aviez fourni ces fichiers objets directement.
- ✓ Vous pouvez créer une archive en utilisant la commande **ar**.
- ✓ Les fichiers archives utilisent l'extension **.a**

\$ ar cr libtest.a test1.o test2.o

☞ combiner test1.o et test2.o dans une seule archive libtest.a

- ✓ Une bibliothèque a un emplacement visible :
 - ☞ /usr/local/lib si la librairie est susceptible d'être utilisée par plusieurs utilisateurs ;
 - ☞ ~/lib si la librairie est susceptible d'être utilisée par un seul utilisateur.
 - ☞ La variable « LD_LIBRARY_PATH » pour les bibliothèques dynamiques

LES BIBLIOTHÈQUES PARTAGÉES

✓ Différences avec les archives :

- ☞ l'édition de liens se fait pendant l'exécution,
- ☞ la réduction de la taille d'un exécutable,
- ☞ la mise à jour de la bibliothèque sans refaire l'édition de liens,
- ☞ Les bibliothèques partagées (shared) ne sont chargées qu'une seule fois en mémoire.

LES BIBLIOTHÈQUES PARTAGÉES

- ✓ Pour créer une bibliothèque partagée, compiler les objets avec l'option -fPIC

```
% gcc -c -fPIC test1.c
```

- ✓ L'option -fPIC indique au compilateur que vous allez utiliser test1.o en tant qu'élément d'un objet partagé.

- ✓ Puis, vous combinez les fichiers objets au sein d'une bibliothèque partagée, comme ceci:

```
% gcc -shared -fPIC -o libtest.so test1.o test2.o
```

- ✓ L'option -shared indique à l'éditeur de liens de créer une bibliothèque partagée au lieu d'un exécutable ordinaire.

- ✓ Les bibliothèques partagées utilisent l'extension .so, (shared object), le nom commence toujours par lib

- ✓ Lier un programme à une bibliothèque partagée (idem statique)

```
% gcc -o app app.o -L. -ltest
```

EXERCICES

1. Ecrire une fonction `void ersys(char *ch)` ; qui permet en cas d'erreur dans un appel de système (code retour égale à -1) d'obtenir le numéro de l'erreur (valeur de la variable externe `errno`) et le message correspondant à l'erreur (`sys_errlist[errno]` où `sys_errlist` est un tableau externe de pointeurs sur caractère). Le paramètre `ch` pointe sur une chaîne qui sera imprimée avant le message correspondant à l'erreur rencontrée (ce pourra être par exemple le nom de l'appel système ayant provoqué l'erreur). Enfin, la fonction `ersys` provoquera la terminaison du processus appelant avec un code de retour égal à `errno`.

2. Re-écrire la fonction

`char *getenv(char *ch)`

en utilisant la variable `environ` (variable environnement externe) ou la forme générale de l'entête d'un programme principal C; on pourra éventuellement utiliser les fonctions de manipulation des chaînes de caractères .

CONVENTIONS DE LA LIGNE DE COMMANDE GNU/LINUX

- ✓ Les options modifient le comportement du programme, alors que les autres arguments fournissent des entrées
- ✓ Les options peuvent prendre deux formes:
 - ☞ Les options courtes sont formées d'un seul tiret et d'un caractère isolé (-h)
 - ☞ Les options longues sont formées de deux tirets suivis d'un nom composé de lettres majuscules, minuscules et de tirets (-- help, --output foo).
- ✓ il est conseillé d'utiliser les noms préconisés dans les standards de codage.
 - ☞ % info "(standards)User Interfaces"

La fonction *getopt_long*, interprète à la fois les options courtes et longues.

UTILISER GETOPT_LONG

- ✓ Pour utiliser getopt_long, vous devez fournir deux structures de données.
 - ☞ La première est une chaîne contenant les options courtes valables, chacune sur une lettre. Une option qui requiert un argument est suivie par deux-points.
 - Exp : **ho:v** indique que les options valides sont -h, -o et -v, la seconde devant être suivie d'un argument.
 - ☞ Pour indiquer les options longues disponibles, vous devez construire un tableau d'éléments struct option. Chaque élément correspond à une option longue et dispose de quatre champs :
 - le nom de l'option longue
 - 1 si l'option prend un argument, 0 sinon;
 - NULL
 - un caractère qui indique l'option courte synonyme de l'option longue.
 - Tous les champs du dernier élément doivent être à zéro.

Forme courte	Forme longue	Fonction
-h	--help	Affiche l'aide mémoire et quitte
-o nom fichier	--output nom fichier	Indique le nom du fichier de sortie
-v	--verbose	Affiche des messages détaillés

EXAMPLE

```
const char* const short_options = "ho:v";
const struct option long_options[] = {
    { "help",      0, NULL, 'h' },
    { "output",    1, NULL, 'o' },
    { "verbose",   0, NULL, 'v' },
    { NULL,        0, NULL, 0   }
};
...
next_option = getopt_long (argc, argv,
    short_options,
    long_options, NULL);
```

INTERACTION AVEC L'ENVIRONNEMENT

- ✓ L'environnement est une collection de paires variable/valeur (sous forme de chaîne de caractères) dans une variable « environ »
- ✓ **char * getenv(const char *name);** /*obtenir la valeur de la variable d'environnement name*/
- ✓ **int setenv(const char *name, const char value, int overwrite);** /*positionner une variable*/
- ✓ **int putenv(char *string);** /*ajouter une variable string est de la forme name=value*/
- ✓ **int unsetenv(const char *name);** /*supprime une variable*/

AFFICHER L'ENVIRONNEMENT D'EXECUTION

PRINT-ENV.C

```
#include <stdio.h>
/* La variable ENVIRON contient l'environnement. */
extern char** environ;
int main ()
{
    char **var;
    for (var = environ; *var != NULL; ++var)
        printf ("%s\n", *var);
    return 0;
}
```

- ✓ Ne modifiez pas environ vous-même; utilisez plutôt les fonctions **setenv** et **getenv**.

PAGES DE MANUEL

- ✓ Les pages de manuel sont divisées en sections numérotées; pour les programmeurs, les plus importantes sont celles-ci:
 - ☞ (1) Commandes utilisateur
 - ☞ (2) Appels système
 - ☞ (3) Fonctions de la bibliothèque standard
 - ☞ (8) Commandes système/d'administration

- ☞ % man sleep
- ✓ la page de manuel de la fonction sleep de la bibliothèque standard, utilisez cette commande:
 - ☞ % man 3 sleep
- ✓ Chaque page de manuel comprend un résumé sur une ligne de la commande ou fonction.
- ✓ whatis liste toutes les pages de manuel (de toutes les sections) pour une commande ou une fonction
- ✓ recherche par mot-clé sur les résumés
 - ☞ man -k mot-clé.

INFO

- ✓ Info contient des informations plus détaillées pour beaucoup de composants fondamentaux du système GNU/Linux et quelques autres programmes.
- ✓ Les pages Info sont des documents hypertextes, similaires aux pages Web.
- ✓ Pour lancer le navigateur texte Info, tapez simplement info à l'invite de commande. Parmi les documents Info les plus utiles, on trouve:
 - ☞ gcc Le compilateur gcc
 - ☞ libc La bibliothèque C GNU, avec beaucoup d'appels système
 - ☞ gdb Le débogueur GNU
 - ☞ emacs L'éditeur de texte Emacs
 - ☞ info Le système Info lui-même